CHEAT SHEET TÉCNICO · JAVAFX + MVC

Estrategias para Eliminar Registros

Guía directa y concisa: lógica, métodos nativos de JavaFX y código exacto por capas.

1. Vista (View)

Detecta la interacción (fila seleccionada, clic en botón o tecla) y se la pasa al Controlador.

2. Controlador (Controller)

Valida que el objeto no sea `null` y le ordena al Manager ejecutar la baja.

3. Manager (Model)

Ejecuta `lista.remove(objeto)`. Al ser `ObservableList`, la tabla se actualiza sola.

💡 **Navegación:** Usa las teclas ← y → o los botones inferiores para revisar cada estrategia.

FUNDAMENTO DE DATOS

0. El Secreto: ¿Por qué la TableView se refresca sola?

Si usas `ObservableList`, no necesitas recargar escenas ni redibujar ventanas manualmente.

LÓGICA A IMPLEMENTAR

1. Enlace en la Vista

`tabla.setItems(controller.obtenerLista`
conecta la vista directamente con la lista observable.

2. Mutación en el Manager

Al hacer `lista.remove(objeto)`, JavaFX dispara un evento interno de redibujo automático.

MÉTODOS INDISPENSABLES:

- `FXCOLLECTIONS.OBSERVABLEARRAYLIST()`
- `LISTA.REMOVE(ITEM)`
- `TABLA.SETITEMS(LISTA)`

VehiculoManager.java & VehiculoController.java — MANAGER & CONTROLLER

```
// --- EN EL MANAGER (Capa Modelo) ---
public class VehiculoManager {
    private ObservableList<Vehiculo> lista;

    public VehiculoManager() {
        this.lista = FXCollections.observableArrayList();
    }

    // Método que elimina de la memoria: ¡dispara el redibujo automático!
    public boolean eliminar(Vehiculo item) {
        return this.lista.remove(item);
    }
}

// --- EN EL CONTROLADOR (Capa Controller) ---
public boolean eliminarVehiculo(Vehiculo item) {
    if (item == null) return false;
    return manager.eliminar(item);
}
```

ESTRATEGIA 1 · RECOMENDADA PARA LA EVALUACIÓN

Fila Seleccionada en Tabla + Botón Exterior

El usuario pincha una fila en la tabla y presiona el botón `[ Eliminar ]`.

LÓGICA PASO A PASO

1. Capturar selección

Usa `getSelectionModel().getSelectedItem()` de la tabla.

2. Validar que no sea null

Si el usuario pulsó el botón sin seleccionar fila, avisa con una alerta o ignora sin caídas.

3. Delegar al Controlador

Envía el objeto obtenido al controlador.

MÉTODOS CLAVE:

`TABLA.GETSELECTIONMODEL().GETSELECTEDITEM`

`CONTROLLER.ELIMINAR...(ITEM)`

CatalogoView.java — VISTA

```
Button btnEliminar = new Button("  Eliminar Seleccionado");

btnEliminar.setOnAction(e → {
    // 1. Obtener el registro de la fila seleccionada
    Vehiculo seleccionado = tabla.getSelectionModel().getSelectedItem();

    // 2. Validación preventiva contra NullPointerException
    if (seleccionado == null) {
        Alert alerta = new Alert(Alert.AlertType.WARNING);
        alerta.setTitle("Atención");
        alerta.setHeaderText(null);
        alerta.setContentText("Debes seleccionar una fila en la tabla primero.");
        alerta.showAndWait();
        return;
    }

    // 3. Ejecución a través del flujo MVC
    controller.eliminarVehiculo(seleccionado);
});
```

ESTRATEGIA 2 · AVANZADA

Botón "Eliminar" en Cada Fila (TableCell)

Una columna con un botón rojo por cada celda. No requiere seleccionar la fila previamente.

LÓGICA PASO A PASO

1. Columna tipo Void

La columna no contiene texto, contendrá un botón.

2. CellFactory personalizada

Se crea un botón en el constructor de la celda y se le asigna el evento.

3. getIndex() para la fila

`getTableView().getItems().get(getInde`
obtiene el objeto de esa fila exacta.

MÉTODOS CLAVE:

- `COL.SETCELLFACTORY(...)`
- `GETINDEX()`
- `SETGRAPHIC(EMPTY ? NULL : BTN)`

CatalogoView.java — Columna con botón

VISTA

```
TableColumn<Vehiculo, Void> colAccion = new TableColumn<>("Acción");

colAccion.setCellFactory(param -> new TableCell<>() {
    private final Button btnFila = new Button("▨ ");

    {
        btnFila.setStyle("-fx-background-color: #ef4444; -fx-text-fill: white; -fx-cursor: hand;");
        btnFila.setOnAction(event -> {
            // Obtener el objeto perteneciente a la celda clickeada
            Vehiculo item = getTableView().getItems().get(getIndex());
            controller.eliminarVehiculo(item);
        });
    }

    @Override
    protected void updateItem(Void item, boolean empty) {
        super.updateItem(item, empty);
        setGraphic(empty ? null : btnFila);
    }
});

tabla.getColumns().add(colAccion);
```

ESTRATEGIA 3 · FORMULARIO ALTERNATIVO

Selector Desplegable (ComboBox / ChoiceBox)

El usuario elige el registro desde un `ComboBox` y pulsa `[ Dar de Baja ]`.

LÓGICA PASO A PASO

1. Enlazar ComboBox a la lista

Se le pasa la misma lista del controlador al constructor del `ComboBox`.

2. Formatear texto visible

Usa `setConverter` o `setCellFactory` para mostrar marca/modelo en vez de `toString()`.

3. Capturar con getValue()

Lee `combo.getValue()` y limpia la selección tras eliminar.

MÉTODOS CLAVE:

- `COMBO.GETVALUE()`
- `COMBO.SETCONVERTER(...)`
- `COMBO.GETSELECTIONMODEL().CLEARSELECTION(`

CatalogoView.java — Selector de baja — VISTA

```
// 1. ComboBox que comparte la misma ObservableList
ComboBox<Vehiculo> combo = new ComboBox<>(controller.obtenerLista());
combo.setPromptText("Selecciona para eliminar...");

// 2. Personalizar la etiqueta en el desplegable
combo.setConverter(new StringConverter<>() {
    @Override
    public String toString(Vehiculo item) {
        return (item == null) ? "" : "#" + item.getId() + " - " + item.getMarca() + " " + item.getModelo();
    }
    @Override
    public Vehiculo fromString(String s) { return null; }
});

// 3. Botón que lee la opción seleccionada
btnEliminarCombo.setOnAction(e -> {
    Vehiculo seleccionado = combo.getValue();
    if (seleccionado != null) {
        controller.eliminarVehiculo(seleccionado);
        combo.getSelectionModel().clearSelection();
    }
});
```

ESTRATEGIA 4 · PROFESIONAL DE ESCRITORIO

Menú Contextual (Clic Derecho en Fila)

Clic derecho sobre la fila abre un `ContextMenu` flotante con la acción de eliminar.

LÓGICA PASO A PASO

1. Configurar RowFactory

Se define cómo se comporta cada fila (`TableRow`) de la tabla.

2. Crear ContextMenu

Se crea el menú con un `MenuItem("Eliminar")` .

3. Enlazar a row.emptyProperty()

Evita que el menú aparezca si el usuario hace clic derecho en el espacio vacío de la tabla.

MÉTODOS CLAVE:

- `TABLA.SETROWFACTORY(...)`
- `ROW.GETITEM()`
- `ROW.EMPTYPROPERTY()`

CatalogoView.java — RowFactory con ContextMenu — VISTA

```
tabla.setRowFactory(tv -> {
    TableRow<Vehiculo> row = new TableRow<>();
    ContextMenu menu = new ContextMenu();
    MenuItem itemEliminar = new MenuItem("🗑 Eliminar este registro");

    itemEliminar.setOnAction(e -> {
        Vehiculo item = row.getItem();
        if (item != null) {
            controller.eliminarVehiculo(item);
        }
    });

    menu.getItems().add(itemEliminar);

    // Asignar el menú únicamente si la fila contiene datos reales
    row.contextMenuProperty().bind(
        Bindings.when(row.emptyProperty()).then((ContextMenu) null).otherwise(menu)
    );

    return row;
});
```

ESTRATEGIA 5 · TECLADO / PRODUCTIVIDAD

Atajo de Teclado (Tecla Supr / DELETE)

El usuario selecciona la fila y presiona la tecla `Delete` (o `Backspace`) para borrar.

LÓGICA PASO A PASO

1. Escuchar eventos de tecla

Se asigna `setOnKeyPressed` al componente `TableView`.

2. Evaluar KeyCode

Se compara `event.getCode() == KeyCode.DELETE`.

3. Tomar selección y borrar

Si hay un elemento seleccionado, se delega al controlador.

MÉTODOS CLAVE:

- `TABLA.SETONKEYPRESSED(...)`
- `EVENT.GETCODE() == KEYCODE.DELETE`

CatalogoView.java — Listener de Tecla — VISTA

```
import javafx.scene.input.KeyCode;


tabla.setOnKeyPressed(event → {
    // Detectar si el usuario presionó la tecla Supr o Backspace
    if (event.getCode() == KeyCode.DELETE || event.getCode() == KeyCode.BACK_SPACE) {
        Vehiculo seleccionado = tabla.getSelectionModel().getSelectedItem();

        if (seleccionado ≠ null) {
            controller.eliminarVehiculo(seleccionado);
        }
    }
});
```

BUENA PRÁCTICA UX

Confirmación Previa con Alert

Evita que el usuario borre registros por error. Compatible con cualquiera de las 5 estrategias.

LÓGICA PASO A PASO

1. Alert CONFIRMATION

Crea una ventana modal nativa con botones Aceptar y Cancelar.

2. showAndWait()

Pausa la ejecución de forma síncrona esperando la respuesta del usuario.

3. Comparar con ButtonType.OK

Solo procede con la baja si el usuario confirmó con Aceptar.

MÉTODOS CLAVE:

- `NEW ALERT(ALERTTYPE.CONFIRMATION)`
- `ALERT.SHOWANDWAIT()`
- `RESPUESTA.GET() == BUTTONTYPE.OK`

Snippet reutilizable con confirmación — UX / VISTA

```
import javafx.scene.control.Alert;
import javafx.scene.control.ButtonType;
import java.util.Optional;

// Crear cuadro de confirmación
Alert confirm = new Alert(Alert.AlertType.CONFIRMATION);
confirm.setTitle("Confirmar Eliminación");
confirm.setHeaderText("¿Estás seguro de eliminar este registro?");
confirm.setContentText("Se eliminará: " + seleccionado.getMarca() + " " + seleccionado.getModelo());

// Esperar respuesta del usuario
Optional<ButtonType> respuesta = confirm.showAndWait();

if (respuesta.isPresent() && respuesta.get() == ButtonType.OK) {
    controller.eliminarVehiculo(seleccionado);
}
```

RESUMEN TÉCNICO

Tabla Comparativa de las 5 Estrategias

Elige la opción adecuada según los requerimientos de tu proyecto o evaluación.

Estrategia	Método Principal JavaFX	Líneas en Vista	Dificultad	Recomendada para
1. Fila + Botón Exterior	`tabla.getSelectionModel().getSelectedItem()`	~6 líneas	⭐ Baja	**Pruebas y Evaluaciones (EP2)**
2. Botón en cada Fila	`TableColumn.setCellFactory(TableCell)`	~20 líneas	⭐⭐⭐ Media	Dashboards y paneles de administración
3. ComboBox Selector	`combo.getValue()`	~12 líneas	⭐⭐ Media	Formularios de bajas independientes
4. Menú Contextual (Clic Derecho)	`tabla.setRowFactory(TableRow & ContextMenu)`	~15 líneas	⭐⭐⭐ Media	Aplicaciones de escritorio avanzadas
5. Tecla Supr / DELETE	`tabla.setOnKeyPressed(KeyCode.DELETE)`	~8 líneas	⭐ Baja	Atajo de teclado rápido (complemento a 1)